

INTO MONSERRATE

Documento Técnico

Computación Visual — Grupo 5 — Semestre 2026-I

Curso	Computación Visual — Grupo 5
Semestre	2026-I
Equipo	Joan Sebastian Roberto Puerto · Baruj Vladimir Ramírez Escalante · Diego Alberto Romero Olmos · Maicol Sebastian Olarte Ramirez · Jorge Isaac Alandete Díaz
Motor	Unity 6 (6000.3.17f1) — Universal Render Pipeline
Lenguaje	C# / .NET (Unity Scripting)

1. Descripción General

Into Monserrate es un videojuego de terror en primera persona (FPS survival-horror) desarrollado en Unity. El jugador recorre un entorno boscoso inspirado en el Cerro Monserrate recolectando tres cruces religiosas mientras evade a una entidad hostil basada en Slenderman. Al reunir todas las cruces, el enemigo desaparece y se habilita la zona de salida para ganar la partida.

2. Herramientas Utilizadas

Herramienta / Paquete	Versión / Detalle	Uso principal
Unity Editor	6000.3.17f1 (Unity 6)	Motor de juego principal
Universal Render Pipeline (URP)	Incluido en Unity 6	Pipeline de renderizado (PC y Mobile)
Cinemachine	Vía Unity Modules	Cámara virtual en primera persona
Unity Input System (New)	PlayerInputActions	Manejo de entradas — teclado y ratón
NavMesh (Unity AI)	com.unity.modules.ai	Navegación autónoma del enemigo
TextMesh Pro	Incluido en proyecto	UI de texto — contador de cruces
CharacterController	com.unity.modules.physics	Movimiento del jugador
Unity Terrain	com.unity.modules.terrain	Escenario con 9 fragmentos de terreno
Unity Particle System	com.unity.modules.particlesystem	Efecto visual de teletransporte del enemigo
Unity Audio	com.unity.modules.audio	Pasos, música de menú, atmósfera, estática
C# / Visual Studio / VS Code	.NET (Unity scripting)	Programación de todos los sistemas

Herramienta / Paquete	Versión / Detalle	Uso principal
Assets externos (Free)	AllSkyFree, Forst, JaneArtWork, YughuesFreeBushes, Tree_Packs	Arte, vegetación, skybox
Modelos FBX personalizados	monserrate.fbx, Slenderman	Modelos 3D propios / adaptados
FreeSound.org	CC / Attribution 4.0	Sonidos ambientales y de UI
Git / GitHub	Control de versiones	Trabajo colaborativo y ramas por funcionalidad

3. Descripción del Flujo Implementado

3.1 Flujo General del Juego

El juego está compuesto por tres escenas Unity conectadas de la siguiente manera:

```

MainMenu.unity → (botón Play)
├─ MainLevel.unity
│   └─ Recolectar 3 cruces → Slenderman activo (persecución +
│       teletransporte)
│       └─ Si el jugador es atrapado → recarga MainLevel (derrota)
│           └─ Cruces completas → Slenderman se desactiva
│               └─ Brújula apunta a la salida → VictoryZone trigger
│                   └─ VictoryScene.unity (victoria)

```

3.2 Movimiento del Jugador — PlayerController.cs

Utiliza CharacterController con gravedad manual. La cámara se gestiona mediante un CinemachineVirtualCamera anclado a un objeto CameraTarget posicionado a 1.6 m de altura sobre el jugador. La rotación horizontal gira el transform completo (yaw); la rotación vertical mueve solo el CameraTarget (pitch), limitado a $\pm 80^\circ$. El input proviene de PlayerInputHandler, que envuelve el nuevo Unity Input System.

3.3 Sistema de Interacción — PlayerInteraction.cs / IInteractable.cs

Se implementó el patrón interfaz IInteractable (polimorfismo): cualquier objeto del mundo implementa Interact() y GetPrompt(). PlayerInteraction lanza un Raycast desde la cámara; si impacta un IInteractable dentro de 3 m y el jugador presiona la tecla de interacción, se invoca Interact(). Objetos interactuables: PickupItem (cruces), SummonHelp (activa la brújula), TriggerZone (activa/desactiva GameObjects al entrar).

3.4 Sistema de Recolección y Progresión — PickupItem.cs / ObjectCounter.cs

Al recoger una cruz, PickupItem.Interact() ejecuta tres acciones en cadena:

- SlendermanController.RegisterCollectedObject() — incrementa la progresión del enemigo.
- ObjectCounter.Instance.CollectItem() — actualiza el contador UI (0/3 Cruces).
- CompassOrbit.DelayedRefresh() — la brújula reapunta al siguiente coleccionable.

Al completar las 3 cruces, el enemigo se desactiva y 4 segundos después la brújula redirige a la zona de salida.

3.5 Sistema de Enemigo — SlendermanController.cs

Navega usando NavMeshAgent. Los comportamientos principales son:

- Detección por línea de visión: HasLineOfSight() comprueba si el jugador está dentro del detectionRange (15 m) y sin obstáculos entre ambos.
- Persecución activa: cuando el jugador es visible, el agente lo sigue sin frenar (autoBraking = false).
- Teletransporte aleatorio: sin ver al jugador durante 40 s, aparece con 80% de probabilidad detrás del jugador (a 2.5 m) y 20% delante (a 10 m), con efecto de partículas y sonido.
- Captura: si la distancia ≤ 2 m, ejecuta CatchPlayer() — recarga la escena (derrota).
- Progresión dinámica: con cada cruz recolectada el enemigo incrementa velocidad y agresividad.

3.6 Efecto de Cordura — SlendermanDetector.cs / SanityEffect.cs

SlendermanDetector (en la cámara del jugador) determina si el jugador mira al Slenderman combinando cuatro condiciones: distancia < 40 m, ángulo dentro del FOV, sin obstáculos (Linecast), y cálculo de lookIntensity $\in [0, 1]$ (1 = mirada directa, 0 = borde del FOV). SanityEffect incrementa el alfa de un panel de estática en el Canvas proporcional al lookIntensity. Si el alfa llega a 1, hay derrota con delay de 1.2 s. Al no mirar, el alfa recupera gradualmente.

3.7 Brújula 3D — CompassOrbit.cs

Objeto 3D hijo del jugador que orbita en el plano horizontal a radio y altura configurables. Apunta al objetivo (tag) más cercano usando interpolación angular suavizada. Al completar la colección, ObjectCounter llama compass.SetTarget(ExitPoint) para redirigir al punto de salida.

4. Dificultades Encontradas

#	Dificultad	Solución aplicada
1	Cámara FPS con Cinemachine — el modo LookAt generaba comportamientos inesperados con la rotación.	Se eliminó el LookAt y se implementó rotación manual del CameraTarget adjunto al jugador, dejando la cámara como hijo directo.
2	NavMesh sobre terreno irregular — el agente se quedaba atascado en los bordes de los 9 tiles de terreno.	Se ajustó la capa groundLayer del Raycast de teletransporte y se configuró NavMeshAgent.autoBraking = false.
3	Efecto de cordura — detectar si el jugador mira directamente a Slenderman requería precisión sin usar la cámara de renderizado.	Se combinó producto punto + ángulo real de FOV + Linecast para calcular lookIntensity con gradiente de intensidad.
4	Teletransporte a posiciones inválidas — Slenderman podía aparecer dentro de la geometría.	Se añadió un Raycast vertical desde altura (raycastHeight = 20f) hacia abajo para verificar suelo válido antes de teletransportar.
5	Sincronización del contador UI con el estado del enemigo — la lógica de	Se centralizó en ObjectCounter (Singleton) que coordina UI, desactivación del enemigo y redirección de la brújula.

#	Dificultad	Solución aplicada
	victoria estaba dispersa en múltiples scripts.	
6	Construcción del escenario 3D — modelar un entorno boscoso reconocible requería assets coherentes y buen rendimiento.	Se usaron assets gratuitos (AllSkyFree, Forst, YughuesFreeBushes), 9 fragmentos de Terrain de Unity y un modelo FBX personalizado (monserrate.fbx).

5. Scripts del Proyecto

A continuación, se listan los scripts C# del proyecto con sus responsabilidades:

Archivo	Responsabilidad
PlayerController.cs	Movimiento FPS, gravedad, rotación cámara, audio de pasos
PlayerInputHandler.cs	Abstracción del nuevo Input System (InputActionAsset)
PlayerInteraction.cs	Raycast de interacción, despacho a IInteractable
IInteractable.cs	Interfaz común para todos los objetos interactivables
SlendermanDetector.cs	Detección FOV + oclusión, cálculo de lookIntensity
SanityEffect.cs	Efecto de pantalla (estática, audio, derrota) ligado a la mirada
SlendermanController.cs	NavMesh AI, persecución, teletransporte, captura, progresión dinámica
PickupItem.cs	Recogida de cruces, notifica contador y enemigo
ObjectCounter.cs	Singleton: progresión de colección, UI, lógica de victoria
CompassOrbit.cs	Brújula 3D orbital hacia el objetivo más cercano
SummonHelp.cs	Activa la brújula al interactuar con el objeto
TriggerZone.cs	Activa/desactiva GameObjects por collider trigger
VictoryZone.cs	Detecta llegada del jugador con cruces completas → carga VictoryScene
MainMenu.cs	Lógica del menú principal (Play, Quit)

Links

Build del proyecto: <https://drive.google.com/file/d/12XavWFNarfVYmKjf04fYGfGA8X7lYb31/view?usp=sharing>

Link repositorio del grupo: <https://github.com/Baruj-Ramirez/CompuacionVisualGrupo5>

Video de presentación del proyecto: https://youtu.be/OnXp_4KbQAc